

Array references to that class.

① Primitive

int [] arr = new int[10];

↓

② Object

Employee [] arr = new Employee[10];

Issues Array

- ① Homogeneous (Same data type);
- ② Fixed size
- ③ Pre defined size
- ④ Cannot grow on demand
- ⑤ May cause memory wastage
- ⑥ not created using any standard structure
△ we don't have supporting methods

→ Collection Framework

- * Can grow on demand
 - * don't have fixed size
 - * size not pre-defined
 - * no wastage of memory.
- a) Represent a group of objects as a single entity.
 - b) Hold Heterogeneous (mixed) → Homogeneous
 - c) Standard data structures
 - d) supporting methods

Comparison

Arrays

- ① Size → Fixed
- ② Cannot grow on demand
- ③ Wastage of memory
- ④ Homogeneous
- ⑤ Higher performance
- ⑥ no underlying data structure
- ⑦ primitive & object
- ⑧ no support of methods

Collection

- ① Size → not fixed
- ② grow on demand
- ③ no wastage of memory
- ④ Homogeneous / Heterogeneous
- ⑤ slower
- ⑥ built standard d/s
- ⑦ objects (no primitive possible)
- ⑧ supporting methods available

Collection Framework.

a) Classes & Interfaces

↓
store, manipulate & traverse
through a group of objects

C++ → STL

Key interface

- a) Collection
- b) List
- c) Set
- d) SortedSet
- e) NavigableSet ≥ 6.0
- f) Queue
- g) Map
- h) SortedMap
- i) Navigable Map

Collection Interface

- a) Defines basic common methods
 - i) add
 - ii) remove
 - iii) contains
 - iv) isEmpty
- ** b) There is no concrete class that directly implements Collection interface

Collection

- a) interface
- b) group of objects as single entity & provides common fn. like add, remove ...

Collections

- a) class
- b) Utility class
Sorting, Searching over a collection

Cursor

a) is an interface used on Collection that allows you to retrieve elements 1 by 1.

- 1) Enumeration { legacy }
- 2) **Iterator** { universal }
- 3) ListIterator

① Enumeration (1.0)

a) Legacy interface (1.0)

b) Legacy classes

c) retrieve data/elements but removal is not possible

d) elements() → Vector ←

Methods of Enumeration
e) hasMoreElements(); ← false

← f) nextElement(); ←

Iterator (universal cursor)

- 1) Universal Cursor
- 2) Retrieve & remove
- 3) On Collection class I can obtain the cursor
iterator();
- 4) Methods.
 - a) hasNext();
 - b) next();
 - c) remove();

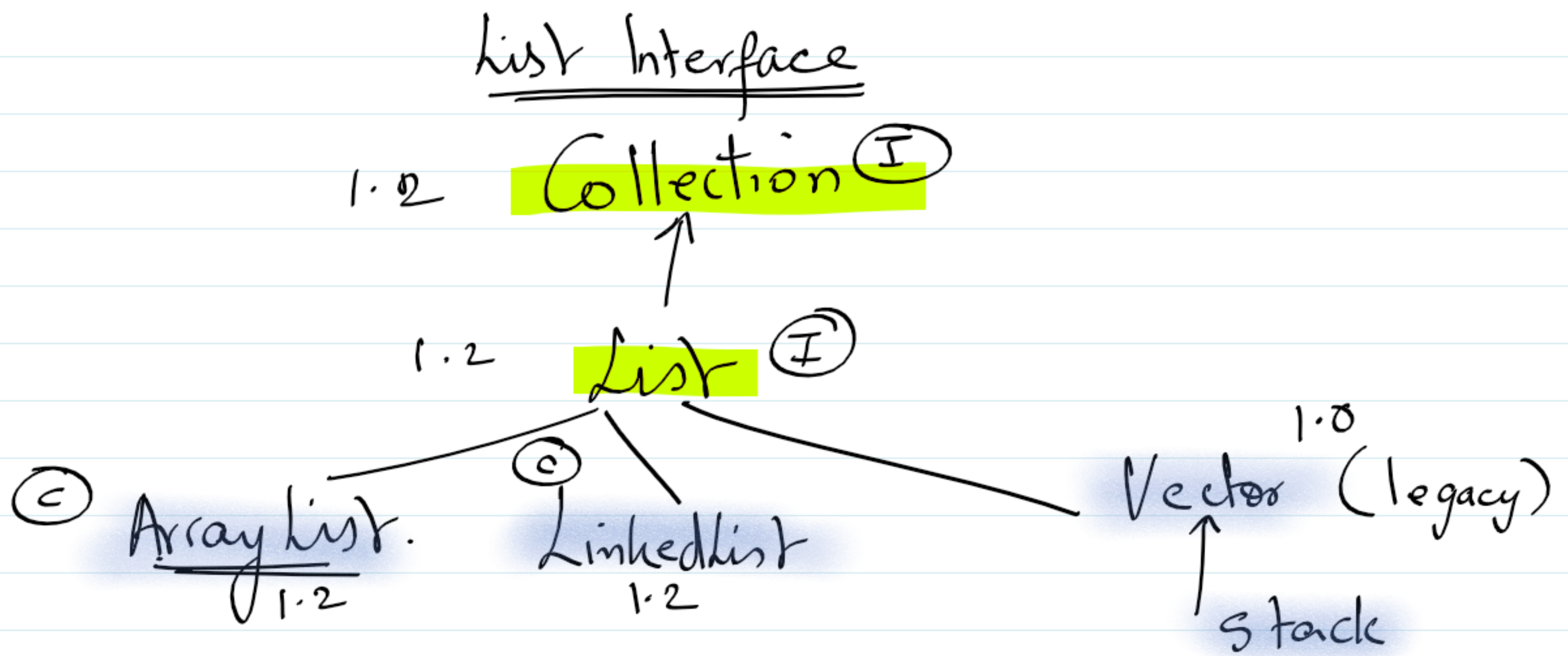
Drawbacks → Cursor Iterator

- ① Forward only direction
- ② Replacing capability absent
- ③ insertion or addition is not possible

List Iterator

- a) Bi-directional Cursor
- b) is only available List →
- c) Read, Remove, Add, Replace
- d) obtained on a list → ListIterator();

	Enumeration	Iterator	LinkedList
① <u>Movement</u>	Only Forward	Only Forward	Bi-directional
② Applicable	only Legacy classes	Universal All Collection Classes	Only list classes
③ Functionality	Read/Retrieval	Read, Remove	Read, Remove Add & Replace
④ How to obtain	elements();	iterator();	LinkedList();
⑤ Methods	hasNextElement(); nextElement();	hasNext(); next(); remove();	⑨ hasNext next hasPrevious previous remove add replace
⑥ Is Legacy	Yes (1.0)	No (1.2)	No (1.2)



- a) Used to represent a group of Objects as a single entity
- b) Duplicate value are allowed
- c) insertion order is preserved.

ArrayList $\rightarrow$ list

a) underlying data structure Growable/Resizable Array.

- b) Duplicate allowed
- c) Insertion order preserved
- d) Hetrogeneous/Homogeneous objects
- e) null insertion is allowed

Drawbacks AL

① 10 + Array $\rightarrow$ n+1
 $\rightarrow$ ② 10 $\rightarrow$ ~~new Array~~ 

Templates C++

Generics

Java

class Arraylist

{

Integer data;

methods

add
remove
+ traverse
linked list

}

class Arraylist
{
String data;

};

class Arraylist

{
Employee data

};

};

class Arraylist < T, U >

U ← String
T ← data;

methods

① ArrayList<String> list = new ArrayList<String>();

② ArrayList<Employee> - - - -

③ AL<Integer> - - - -

LinkedList

prev ← [] → next

- (a) underlying data structure is Doubly List
- (b) Heterogeneous objects
- (c) Insertion order preserved
- (d) Duplicates are allowed
- (e) null is allowed
- (f) implements — Serializable, Cloneable X Random Access
- (8) best choice frequent add, deletion or insertion
- (9) worst choice frequent traversing

